```
import java.util.*;

class Node {
    int data;
    LinkedList<Node> neighbors;

    public Node(int data) {
        this.data = data;
        this.neighbors = new LinkedList<>();
    }

    public void addNeighbor(Node neighbor) {
        neighbors.add(neighbor);
    }
}

public class BFS {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Get number of nodes and create node list
        System.out.println("Enter number of nodes:");
        int numNodes = scanner.nextInt();
        List<Node> nodes = new ArrayList<>();
        for (int i = 0; i < numNodes; i++) {
            nodes.add(new Node(i));
        }

        // Get graph connections from user input
        System.out.println("Enter connections (node1 node2):");
        while (scanner.hasNextInt()) {
            int node1 = scanner.nextInt();
            int node2 = scanner.nextInt();
            nodes.get(node1).addNeighbor(nodes.get(node2));
```

```
    }

    // Get source node for BFS
    System.out.println("Enter source node:");
    int sourceNode = scanner.nextInt();

    // Perform BFS
    bfs(nodes, nodes.get(sourceNode));
}

private static void bfs(List<Node> nodes, Node source) {
    boolean[] visited = new boolean[nodes.size()];
    Queue<Node> queue = new LinkedList<>();

    // Mark source node as visited and enqueue it
    visited[source.data] = true;
    queue.add(source);

    while (!queue.isEmpty()) {
        Node currentNode = queue.poll();

        // Print current node
        System.out.print(currentNode.data + " ");

        // Explore unvisited neighbors of current node
        for (Node neighbor : currentNode.neighbors) {
            if (!visited[neighbor.data]) {
                visited[neighbor.data] = true;
                queue.add(neighbor);
            }
        }
    }

    System.out.println();
}
```

}